Enunciado S10 · Chatbot con RAG básico, fuentes y rechazo

Sesión: S10 · RAG (jueves 22 de octubre de 2026, 18:30–20:00 `Europe/Madrid`).

Dedicación orientativa del alumno: 4 a 6 horas (práctica autónoma fuera de los 90 minutos de clase).

Objetivo: construir de principio a fin el flujo completo que permite responder con documentación propia, citar las fuentes que respaldan cada respuesta y rechazar antes de generar lo que esa documentación no permite saber.

---

1. Requisitos técnicos obligatorios del ejercicio

1.1 Corpus sintético e ingesta desde índice vacío

- Preparar **al menos ocho documentos ficticios en formato Markdown o TXT** sobre un dominio coherente (por ejemplo, arquitectura, políticas operativas, tarifas, SLA o soporte de un producto ficticio).
- Implementar el flujo completo y reproducible de **lectura desde los archivos en bruto, limpieza, fragmentación (chunking con solapamiento), cálculo de embeddings, almacenamiento e indexación partiendo de un índice vacío**.
- Asignar **identificadores estables** tanto a los documentos como a los fragmentos (por ejemplo, `<version>:<documento>:<ordinal>`) y registrar la versión preparada del corpus.

1.2 Stack, modelos por variables, `vector(768)` e índice HNSW

- Base de datos vectorial: **PostgreSQL con `pgvector`** (`CREATE EXTENSION IF NOT EXISTS vector;`).
- Referencias del documento publicado:
- Modelo generativo configurable mediante la variable `GEN_MODEL_ID` (referencia: `gemini-3.8-flash`).
- Modelo de embeddings configurable mediante la variable `EMBEDDING_MODEL_ID` (referencia: `gemini-embedding-2`).
- Conexión configurable mediante `DATABASE_URL` (sin credenciales escritas en el código).
- **Embeddings de 768 dimensiones**, almacenados en una columna `vector(768)` con un **índice HNSW** explícito:

```
CREATE INDEX ON lab_chunks USING hnsw (embedding vector_cosine_ops) WITH (m = 16, ef_const
```

- Utilizar **exactamente el mismo modelo de embeddings y la misma dimensión (`768`)** al indexar los documentos y al consultar las preguntas, registrando las versiones de corpus, modelo y configuración.

1.3 Búsqueda vectorial, escala de puntuación y control previo a generar

- Consultar los fragmentos más próximos en PostgreSQL/pgvector y **explicar la escala de puntuación**:

- En `pgvector`, el operador `<=>` mide **distancia coseno** en \([0, 2]\), donde **un valor menor significa mayor proximidad** (`0` = vectores idénticos).
- La **similitud coseno** es `1 - (embedding <=> $1::vector(768))`, donde un valor mayor significa mayor proximidad.
- Ni la distancia ni la similitud coseno son una probabilidad estadística de que la respuesta sea verdadera.
- **Aplicar un control de puntuación (umbral) antes de llamar al modelo generativo:**
- Calibrar el umbral con **ejemplos de validación separados del conjunto de test**.
- Si ningún fragmento recuperado supera el umbral calibrado, el sistema debe **rechazar la consulta de forma controlada sin invocar al modelo generativo**.
- Una simple instrucción en el prompt del tipo *«si no sabes la respuesta, dilo»* **no sustituye** este control previo en código.

1.4 Generación fundamentada, citas y protección frente a inyección

- Construir el contexto únicamente con los fragmentos que superaron el umbral e incluir sus identificadores reales.
- **El contenido recuperado es dato no fiable, nunca instrucciones que reconfiguren el chatbot:** delimitar los fragmentos en un bloque de datos y neutralizar u omitir cualquier instrucción incrustada en los documentos.
- La respuesta generada debe respaldar sus afirmaciones citando los fragmentos empleados (`[cite:id]` o formato equivalente resoluble contra el corpus) y filtrar cualquier cita inventada que no pertenezca a la evidencia seleccionada.

---

2. Evaluación obligatoria sobre 20 preguntas (14 respondibles + 6 sin respuesta)

- Diseñar un conjunto de **20 preguntas de test**:
- **14 preguntas respondibles** con la información de vuestros documentos ficticios.
- **6 preguntas sin respuesta** en el corpus (datos ausentes que el sistema debe rechazar).
- **Separación estricta entre validación y test:**
- Los ejemplos usados para ajustar el troceado, el prompt o calibrar el umbral forman el conjunto de **validación**.
- Las **20 preguntas de test** se redactan con su **respuesta esperada (y documento/fragmento fuente) o su motivo de rechazo definidos antes de ejecutar**, y **no se reajusta el sistema después de ver los resultados del test**.
- **Ejecutar la evaluación y presentar los resultados:**
- Guardar por cada una de las 20 preguntas: los fragmentos recuperados, sus puntuaciones, la decisión del control previo, la respuesta final y las fuentes citadas.
- Informar explícitamente los **aciertos sobre 14 respondibles (`X/14`)** y los **rechazos correctos sobre 6 sin respuesta (`Y/6`)**, identificando las versiones de corpus, preguntas y configuración utilizadas.
- *Nota de honestidad:* si por una limitación técnica justificada algún caso quedase pendiente de ejecución, debéis describirlo con transparencia sin inventar cifras, pero tened en cuenta que **una**

evaluación solo preparada y sin ejecutar no completa el requisito de evaluación de la entrega.

---

3. Entrega

1. **Código de reconstrucción/ingesta y de consulta** (ejecutable desde documentos en bruto e índice vacío).
2. **Corpus sintético** (mínimo 8 documentos Markdown o TXT).
3. **Configuración de base de datos PostgreSQL +** `pgvector` (`vector(768)` e índice HNSW).
4. **Resultados ejecutados de la evaluación de 20 preguntas** (`X/14` respondibles y `Y/6` rechazos correctos, con el registro de fragmentos, puntuaciones, respuestas y fuentes por caso).
5. `README.md` con las variables (`GEN_MODEL_ID`, `EMBEDDING_MODEL_ID`, `DATABASE_URL`, umbral calibrado), explicación de la escala y comandos exactos para reconstruir el índice y reproducir la evaluación.

(Opcional: lectura de PDFs o interfaz gráfica; no se exige frontend ni despliegue en la nube).

---

4. Material de apoyo incluido en este repositorio

- **Laboratorio de referencia (`lab/`):** implementación completa en TypeScript y PostgreSQL/pgvector (`lab/schema.sql`, `lab/docker-compose.yml`, 10 documentos ficticios de AetherCloud, `vector(768)`, índice HNSW, calibración con validación y evaluación ejecutada de 20 preguntas).
- **Guía de apoyo del laboratorio y del demostrador web:** consulta `laboratorio-apoyo.md` para los comandos locales y la relación con el demostrador de aula desplegado en Cloudflare.